

Sprint Tales - Sprint 1

Sprint: 1

Período: 07/05/2026 a 13/05/2026

QM: Willian Gomes Pessoa

TT01 - Definir e documentar a stack

Prioridade: Must

Responsável: Willian

Alterado depois: Não

Solução adotada: Stack definida como Python 3.12, Flask 3.0.3, MySQL 8.0 e mysql-connector-python 9.0.0. Flask foi escolhido pela simplicidade e curva de aprendizado adequada; MySQL pelo modelo relacional do domínio; queries SQL diretas (sem ORM) por transparência na revisão técnica.

Arquivos alterados: `docs/stack.md`

Impacto: Documenta a base técnica do projeto para todos os membros do time.

TT02 - Modelar banco de dados

Prioridade: Must

Responsável: Willian

Alterado depois: Não

Solução adotada: Schema SQL criado com 4 tabelas: `usuarios`, `password\_reset\_tokens`, `disciplinas` e `monitorias`. Diagrama ER gerado em Mermaid. Constraints de integridade referencial explícitas (FK, UNIQUE, ON DELETE CASCADE).

Arquivos alterados: `backend/db/schema.sql`, `docs/modelagem-banco.md`

Impacto: Base de dados que suporta todas as sprints seguintes.

TT03 - Configurar ambiente de desenvolvimento local

Prioridade: Must

Responsável: Willian

Alterado depois: Não

Solução adotada: Guia de setup criado com Docker Compose para o MySQL, venv Python, variáveis de ambiente e script de criação do primeiro usuário admin (`scripts/create\_admin.py`).

Arquivos alterados: `docs/setup.md`, `docker-sql.yaml`, `backend/scripts/create\_admin.py`

Impacto: Qualquer membro consegue subir o ambiente do zero sem ajuda verbal.

TT04 - Criar estrutura base do projeto (skeleton)

Prioridade: Must

Responsável: Willian

Alterado depois: Não

Solução adotada: App factory com `create\_app()` e blueprints separados por domínio: `auth`, `usuarios`, `disciplinas`, `agenda`, `registros` e `relatorios`. Rota `GET /health` retornando HTTP 200. Templates e arquivos estáticos servidos de `frontend/`, separados do backend.

Arquivos alterados: `backend/app.py`, `backend/requirements.txt`, estrutura de blueprints

Impacto: Estrutura base da qual todos os módulos seguintes dependem.

TT05 - Configurar conexão da aplicação com MySQL

Prioridade: Must

Responsável: Willian

Alterado depois: Não

Solução adotada: Connection pool MySQL com 5 conexões reutilizáveis via `MySQLConnectionPool`. Todas as configurações (host, porta, usuário, senha, banco) lidas de variáveis de ambiente pela classe `Config`. Nenhuma credencial hardcoded. Erro de conexão na inicialização logado via `logging.exception`.

Arquivos alterados: `backend/db/connection.py`, `backend/config.py`, `backend/.env.example`

Impacto: Integração entre aplicação e banco sem exposição de credenciais no código.

US01 - Admin cadastra usuários com perfis

Prioridade: Must

Responsável: Willian, jpmab26

Alterado depois: Não

Solução adotada: CRUD de usuários com 4 papéis (ALUNO, MONITOR, PROFESSOR,

ADMIN). Admin cria o usuário com status `PENDENTE` e uma senha temporária gerada via `secrets.choice` (10 caracteres alfanuméricos). A senha temporária é exibida ao admin na tela - não enviada por email. Deativação e reset de senha disponíveis na tela de gestão.

Arquivos alterados: `backend/usuarios/service.py`, `backend/usuarios/routes.py`,
`backend/usuarios/repository.py`, `frontend/templates/usuarios/index.html`

Impacto: Base de autenticação e controle de acesso para todos os épicos seguintes.

US02 - Usuário faz login com email e senha

Prioridade: Must

Responsável: Willian, jpmab26

Alterado depois: Não

Solução adotada: Autenticação via email e senha com verificação de hash (werkzeug).

Usuário com status `PENDENTE` é redirecionado para tela de primeiro acesso antes de acessar o sistema. Após troca de senha, status atualizado para `ATIVO`. Sessão por cookie com flag HTTPOnly. Redirecionamento pós-login por papel (ADMIN, MONITOR, PROFESSOR, ALUNO).

Arquivos alterados: `backend/auth/service.py`, `backend/auth/routes.py`,
`backend/auth/repository.py`, `backend/auth/decorators.py`,
`frontend/templates/auth/login.html`, `frontend/templates/auth/first\_access.html`

Impacto: Implementa o fluxo de autenticação completo do EP01.

US03 - Monitor edita seu perfil

Prioridade: Should

Responsável: Willian

Alterado depois: Não

Solução adotada: Monitor autenticado pode atualizar `contato` e `disponibilidade` no próprio perfil. Operação isolada em `update\_monitor\_profile` no service, sem afetar dados de autenticação.

Arquivos alterados: `backend/usuarios/service.py`, `backend/usuarios/routes.py`,
`frontend/templates/usuarios/my\_profile.html`

Impacto: Permite que monitores mantenham seu perfil atualizado no sistema.

US04 - Admin desativa um usuário

Prioridade: Should

Responsável: Willian

Alterado depois: Não

Solução adotada: Admin pode desativar qualquer usuário, alterando status para `INATIVO`. Usuário inativo não consegue autenticar - verificação feita no fluxo de login. Operação disponível na tela de gestão de usuários.

Arquivos alterados: `backend/usuarios/service.py`, `backend/usuarios/routes.py`, `frontend/templates/usuarios/index.html`

Impacto: Controle de acesso por status no EP01.

Explicação da Lógica Técnica - Sprint 1

Elaborado por: Desenvolvedores da Sprint 1

Destinado a: Willian Gomes Pessoa (Quality Manager)

Sprint: 1 - Perfis e Autenticação (07/05 a 13/05/2026)

Visão geral da arquitetura

O backend é uma aplicação Flask organizada em **blueprints** - módulos independentes, um por domínio do sistema. A comunicação entre o browser e o servidor usa **sessão por cookie** (sem JWT). O banco de dados é MySQL, acessado por **queries SQL diretas** (sem ORM).

...

browser

- └─ Flask (app.py)
- └─ auth/ ← login, logout, primeiro acesso
- └─ usuarios/ ← CRUD de usuários, perfis
- └─ disciplinas/ ← (placeholder Sprint 1)
- └─ agenda/ ← (placeholder Sprint 1)
- └─ registros/ ← (placeholder Sprint 1)
- └─ relatorios/ ← (placeholder Sprint 1)
- └─ db/connection.py ← pool de conexões MySQL

...

TT01 - Stack

Componente	Escolha	Motivo
Linguagem	Python 3.12	Familiaridade do time
Framework	Flask 3.0.3	Simples, sem "magia" - cada rota é explícita
Banco	MySQL 8.0	Modelo relacional, SQL auditável
Conector	mysql-connector-python 9.0.0	Oficial da Oracle, sem dependências extras
Queries	SQL direto (sem ORM)	Transparência para revisão técnica e auditoria

TT02 - Modelagem do banco

O schema define 4 tabelas para suportar Sprint 1 e preparar Sprint 2:

``usuarios`` - tabela central. Cada usuário tem um ``papal`` (ALUNO, MONITOR, PROFESSOR, ADMIN) e um ``status`` (PENDENTE, ATIVO, INATIVO). O campo ``senha_temporaria`` indica se o usuário ainda não trocou a senha inicial.

``password_reset_tokens`` - tokens de reset de senha com expiração. Pertence a um usuário via FK com ``ON DELETE CASCADE``.

``disciplinas`` - tabela preparada para Sprint 2 (EP02). Já criada para evitar migration futura.

``monitorias`` - vínculo entre aluno e disciplina. Constraint ``UNIQUE(disciplina_id, aluno_id)`` impede duplicidade.

TT03 - Setup local

O ambiente local usa **Docker Compose** para subir o MySQL sem instalar nada na máquina. O schema é aplicado automaticamente no primeiro start. O backend roda em **venv Python** com dependências fixadas em ``requirements.txt``.

Fluxo de setup em 7 passos documentados em ``docs/setup.md``:

1. Criar venv e instalar dependências
2. Subir MySQL via Docker
3. Configurar variáveis de ambiente (``env.example`` como base)
4. Criar o primeiro usuário ADMIN via script
5. Rodar a aplicação

TT04 - Skeleton do projeto

A aplicação usa o padrão **app factory** (``create_app()``): a instância Flask é criada dentro de uma função, o que facilita testes e configuração. Cada domínio é um **blueprint** registrado no

factory.

A rota `GET /health` retorna `{ "status": "ok" }` com HTTP 200 - usada para verificar se a aplicação está no ar.

Templates e arquivos estáticos (CSS, JS) estão em `frontend/`, separados do backend.

TT05 - Conexão com MySQL

A conexão usa **connection pool** com 5 conexões reutilizáveis - evita abrir e fechar conexão a cada request. Todas as configurações (host, porta, usuário, senha, banco) são lidas de **variáveis de ambiente** via a classe `Config`. Nenhuma credencial está hardcoded no código.

Se o banco não estiver disponível na inicialização, o erro é **logado no console** com `logging.exception` - a aplicação não sobe silenciosamente com banco ausente.

US01 - Cadastro de usuários com perfis

Fluxo principal (admin cria usuário):

1. Admin preenche nome, email e papel no formulário
2. Backend valida: papel deve ser ALUNO/MONITOR/PROFESSOR/ADMIN; nome e email obrigatórios; email não pode ser duplicado
3. Senha temporária é gerada com `secrets.choice` (10 caracteres alfanuméricos - criptograficamente seguro)
4. Hash da senha é armazenado no banco (werkzeug `generate\_password\_hash`)
5. Usuário criado com status `PENDENTE` e `senha\_temporaria=True`
6. A senha temporária é exibida ao admin na tela - não é enviada por email

Outras operações disponíveis:

- Desativar usuário: atualiza status para INATIVO
 - Reset de senha: gera nova senha temporária, exibe ao admin
 - Monitor edita perfil: atualiza contato e disponibilidade no próprio perfil
-

US02 - Login com email e senha

Fluxo de login:

1. Usuário informa email e senha
2. Backend busca usuário por email e verifica com `check\_password\_hash`
3. Se status for `INATIVO`: rejeita com mensagem de conta inativa

4. Se status for `PENDENTE`: redireciona para tela de **primeiro acesso** (troca de senha obrigatória)
5. Se status for `ATIVO`: cria sessão com `user\_id`, `nome`, `apelido` e redireciona por papel:
 - ADMIN → gestão de usuários
 - MONITOR → perfil próprio
 - PROFESSOR/ALUNO → home

Primeiro acesso:

1. Usuário informa nova senha (mínimo 8 caracteres) e confirmação
2. Backend atualiza hash no banco e muda status para `ATIVO`
3. Usuário é redirecionado ao dashboard

Segurança da sessão:

- Cookie com flag HTTPOnly (não acessível por JavaScript)
- Em produção, flag Secure ativado via variável de ambiente
- Sessão expira em 8 horas

Pontos de atenção para o QM

Ponto	Detalhe
Senha temporária	Exibida uma única vez ao admin - não há recuperação posterior
Email	Normalizado para lowercase antes de salvar e comparar
Status PENDENTE	Usuário com este status não acessa o sistema - é forçado a trocar
Blueprints futuros	`disciplinas`, `agenda`, `registros`, `relatorios` existem como placeholders